자기소개서 — 현대오토에버 계정 시스템 프론트엔드 개발 및 운영

정다훈

항목 1 · 제품의 불확실성을 기술적 의사결정으로 해결하는 Product Engineer

저는 복잡한 요구사항과 기술적 제약 속에서 **제품의 핵심 문제를 정의하고, 적절한 구조와 실행 방향을 결정해 실제 서비스로 완성하는 Product Engineer입니다.** 프론트엔드를 핵심 전문성으로 삼되, 데이터 구조와 사용자 경험, 백엔드와 LLM의 연결까지 제품 전체를 바라보며 의사결정을 주도합니다. 레거시 화면을 Next.js로 전환하고, 대용량 데이터 화면의 렌더링 병목을 찾아 줄이고, 고객사별 테마와 기능을 설정으로 바꾸는 공통 컴포넌트를 만들 때에도 판단의 기준은 사용자가 직접 만나는 화면의 구조와 안정성이었습니다.

한국미스미 프로젝트에서는 Next.js·TypeScript 기술 리더로서 **PHP·jQuery 기반 레거시 시스템의 마이그레이션을 이끌었습니다.** 기존 시스템을 단순히 새로운 기술로 재구현하는 것이 아니라, 운영 중인 기능과 제약을 분석하고 향후 확장과 유지보수가 가능한 개발 구조로 전환했습니다. 삼성물산 프로젝트에서는 대용량 데이터를 처리하는 사내 프로그램을 고도화하며 **데이터 흐름, 성능, 사용자 경험을 함께 고려해 문제를 해결했습니다.** 내담씨앤씨 SI로 수행한 두 프로젝트(삼성물산·한국미스미)에 걸쳐서는 **SSO·OAuth 2.0 인증 연동의 프론트엔드 측을 맡아** Authorization Code 흐름의 리다이렉트·콜백(code/state) 처리와 로그인 화면, 만료된 토큰을 감지해 갱신한 뒤 요청을 다시 보내고 갱신에 실패하면 재로그인으로 전환하는 인터셉터를 구현했고, 콜백의 state를 검증해 CSRF를 막고 Access 토큰은 메모리에, Refresh 토큰은 서버가 설정하는 httpOnly 쿠키에 두도록 연동해 XSS 노출을 줄였습니다.

대웅제약의 AI 제품에서는 시니어 개발자로 참여해 **데이터 구조부터 사용자 화면까지 End-to-End로 책임졌습니다.** 초기 구상 단계에서 요구사항을 기술적으로 구체화하고, 기획·LLM·백엔드 담당자와 함께 각 영역의 제약과 우선순위를 조율했습니다. 단순히 정해진 기능을 구현하는 것이 아니라, 제품이 실제 환경에서 작동하기 위해 필요한 구조와 사용자 흐름을 설계했습니다. 그 결과 **제품은 현재 대웅 임직원 약 3,500명이 사용하는 서비스로 출시됐습니다.**

기술 리더십의 범위도 제품 구현에만 두지 않았습니다. AI를 활용한 Product Engineering 방식을 탐구하고, 팀에 하네스 엔지니어링을 도입했습니다. **AI 활용에 필요한 개발 맥락과 검증 기준을 구조화하고 이를 개발 표준으로 정착시켜, 개인의 경험에 의존하던 방식을 팀이 반복적으로 활용할 수 있는 체계로 발전시켰습니다.**

제 역할은 가장 어려운 코드를 직접 작성하는 것에 그치지 않습니다. 불확실한 상황에서 방향을 결정하고, 여러 직군의 관점을 기술적으로 연결하며, 팀이 더 좋은 제품을 지속적으로 만들 수 있는 기반을 세워야 합니다. 저는 **그 판단과 결과를 끝까지 책임집니다.**

항목 2 · AI의 속도를 팀의 개발 역량으로 전환하다

74개 페이지의 **React 전환을 2주 내에, RESTful API 29건 개발을 1주 내에 완료했습니다.** 단순히 AI로 더 많은 코드를 생성한 것이 아니라, **명세 분석부터 개발·테스트·보안·배포까지 하나의 하네스 엔지니어링 체계로 연결한 결과입니다.**

AI를 개발에 적용하자 새로운 병목이 나타났습니다. 제품의 맥락과 구현 기준을 매번 다시 설명해야 했고, 생성된 코드의 타입 오류와 금지 패턴, 비밀키 노출 가능성을 사람이 반복해서 확인해야 했습니다. 구성원마다 활용 방식이 달라 결과물의 품질에도 편차가 생겼습니다. 프롬프트 작성법을 공유하는 것만으로는 이 문제를 해결할 수 없다고 판단했습니다. **개발 맥락과 규칙을 재사용 가능한 자산으로 만들고, 명확한 기준으로 판단할 수 있는 영역은 시스템이 자동으로 검증하도록 방향을 전환했습니다.**

먼저 디자인과 개발 맥락을 연결하는 MCP 기반 도구를 도입하고, 데이터 시각화 공통 로직과 UI 생성 기준을 SKILL.md와 Design.md로 표준화했습니다. 개발자가 반복해서 설명하던 내용을 AI가 지속적으로 활용할 수 있는 개발 자산으로 바꿨습니다. 그 결과 **신기능 개발 리드타임은 1주에서 1일로, 신규 차트 개발 시간은 1시간에서 10분으로 줄었습니다.**

속도 향상이 품질 저하로 이어지지 않도록 검증 체계도 구축했습니다. E2E 발화 테스트와 LLM 응답 검증, CI/CD를 하나의 흐름으로 통합하고 타입 오류·금지 패턴·비밀키 노출을 코드 편집 단계에서 자동 점검했습니다. **400여 건의 발화 테스트를 2시간 안에 수행하면서 해당 테스트 범위의 출력 오류를 0건으로 유지했고, 기존 4단계의 코드 검증 절차를 1단계로 줄였습니다. UI 회귀 테스트는 24개 시나리오와 248개 스텝을 3개 뷰포트에서 자동화했으며, 테스트 커버리지는 54%에서 84%로 높였습니다.**

이 방식을 개인의 작업 방식에 머물게 하지 않았습니다. 개발 팀원 5명과 반복 오류와 검증 기준을 정리하고, 주 1회 개발자 세미나를 직접 주최·운영해 총 12회 적용 사례를 공유했습니다. 이 과정에서 **48건의 기술 문서를 축적하고 AI 산출물의 형식과 검증 절차를 팀의 공통 기준으로 표준화했습니다.** 작은 차트와 반복 컴포넌트에서 시작한 방식을 **UI 회귀와 LLM 응답 검증을 거쳐 React 전환과 API 개발까지 단계적으로 확장했습니다.**

항목 3 · 사업의 변화를 제품의 구조와 운영 방식으로 전환하다

AI코치를 10주 동안 개발해 실서비스로 완성한 뒤, 고객사마다 제공 기능을 다르게 구성하고 그 차이에 따라 가격을 책정해야 한다는 요구가 생겼습니다. 고객사별 화면을 따로 만들면 첫 납품은 빠르지만, 고객이 늘어날수록 같은 기능을 반복해서 수정하고 검증해야 합니다. 저는 **기능별 가격 정책을 화면 커스터마이징이 아니라, 하나의 서비스에서 여러 상품을 운영하기 위한 제품 설계 문제로 정의했습니다.**

기획·운영 담당자와 고객사별 기능 범위와 제어 기준을 정하고, 완성된 코드의 상당 부분을 다시 설계했습니다. 고객사별 테마와 기능 정책은 Base-Theme·Config-Driven UI로 분리하고, 변경되는 이유가 같은 영역만 공통화했습니다. **고객사별로 화면을 개발하던 방식을, 기획·운영 담당자가 설정값으로 상품을 구성하는 방식으로 전환했습니다.**

새로운 방식은 신규 고객사 웰체크 온보딩에서 검증했습니다. 색상·CI·기능 구성을 설정으로 조합해 별도의 제품을 다시 개발하지 않고 고객사 요구를 반영했습니다. **최초 AI코치 구현 10주 대비, 웰체크 프론트엔드 커스터마이징·납품을 2주 안에 완료했습니다.** 이 기간은 LLM과 백엔드 일정을 제외한 프론트엔드 납품 기준입니다.

AI코치에서 고객사별 기능 구성과 가격 정책을 분리했다면, 비즈36.5에서는 권한·메뉴·브랜딩·데이터 출력 정책까지 고객사별로 구분해야 했습니다. Spring Boot·Thymeleaf·jQuery와 MySQL 쿼리가 강하게 결합된 기존 구조에서는 고객사별 차이가 화면과 서버 로직 곳곳에 섞일 위험이 있었습니다. **하나의 제품을 유지하면서도 고객사마다 다른 사용 범위와 노출 정책을 적용할 수 있도록 경계를 다시 정했습니다.**

운영 중인 서비스를 한 번에 다시 만드는 대신 React를 화면 단위로 적용하고, 공통 기능과 고객사별 정책을 분리했습니다. 사용자 권한·메뉴·브랜딩·데이터 출력 기준을 정리하면서 **108개 페이지·82개 화면을 9주 동안 순차적으로 전환했습니다. 운영 중인 강결합 레거시를 점진적으로 전환해, 고객사별 CI·메뉴·기능 변경을 1주 안에 처리할 수 있게 했습니다.**

두 프로젝트에서 분리한 대상은 달랐습니다. AI코치에서는 고객사에 어떤 기능을 상품으로 제공할 것인지를 분리했고, 비즈36.5에서는 누가 어떤 메뉴와 데이터를 사용할 것인지를 분리했습니다. **개별 고객사의 요청을 코드에 계속 추가하지 않고, 반복되는 사업 요구를 제품의 규칙으로 바꿔 다음 고객사를 수용할 수 있게 했습니다.**